BIG Certificate Portal — Application Data Flow Report

End-to-end movement of data: Browser → React SPA → Express API → PostgreSQL / IPFS / Sepolia

Generated: 2026-09-30 · Source: /home/pradeep/certificate-app · Render: server.js (1970 lines) + src/ (React 19) · Revised: adds Domains & Hosts Used + rationale

0. System Context & Data Domains

DOMAIN	TABLES / STORES	EXTERNAL INTEGRATION
Certificates	`certificates`, `activity`	IPFS (cid), Sepolia chain (txHash)
Results	`results`, in-memory `otpStore` + `app_settings` (sms)	SMS gateway (otp)
Admin & Auth	`users`, `app_sessions`, `app_settings`, `brand`, `departments`	—
Issues / Analytics	`issues`, `activity`	—
Campus Monitor	`monitor_apps`, `monitor_checks`, `monitor_events`	external apps POST `/api/agent/event`
DPIA / Privacy	`dpia_assessments`, `dpia_risks`, `dpia_workflow_config`, `dpia_audit_log`	—

0.1 Domains & Hosts Used

The portal is **domain-agnostic**: in production it is served same-origin and talks only to the hosts/endpoints listed below. Values reflect the current deployment (LAN hosts `.71` and `.85`). No secrets/tokens are listed.

A. Deployment & Runtime Hosts (application domain)

ROLE	DOMAIN / HOST	NOTES
Production server (serves `build/`)	`localhost:5000`	Express, API + static SPA
CRA dev server	`localhost:3000`	React development only
Host `.71` (primary)	`10.19.172.47:5000`	node server, serves build
Host `.85` (secondary)	`192.168.29.85:5000`	PM2 app `cert-app` (auto-restart)
Database	`localhost:5432`	PostgreSQL 16 (local, per host)
Frontend API base	`API_BASE = dev ? http://<hostname>:5000 : ""`	Empty string ⇒ same-origin in production

B. External Service Domains (outbound integrations)

PURPOSE	DOMAIN / ENDPOINT	DIRECTION / DATA
IPFS certificate file storage (Pinata)	`uploads.pinata.cloud` (`/v3/files`) + IPFS network	Outbound: certificate blob → content-addressed `cid`

PURPOSE	DOMAIN / ENDPOINT	DIRECTION / DATA
Blockchain anchoring (Sepolia testnet JSON-RPC)	`ethereum-sepolia.publicnode.com`	Outbound: `addCertificate(hash,name,ipfsHash)` → `txHash`
SMS provider (configurable)	`www.fast2sms.com` · `control.msg91.com` · `api.textlocal.in`	Outbound: OTP SMS (no student data beyond mobile + OTP)
Local SMS gateway (on-device)	App-configured phone URL on the LAN (Termux API, `com.termux.api`)	Outbound: OTP SMS via Android SIM
Monitored campus applications	Per-app URL configured in *Campus Monitor*	Both: poll health; apps POST events with `x-monitor-token`

C. Functional Data Domains (internal)

Certificates · Results · Admin & Auth · Issues / Analytics · Campus Monitor · DPIA / Privacy — see the table in Section 0 for their tables/stores. These are bounded contexts inside the single PostgreSQL database, not network domains.

THOUGHT / RATIONALE

The application deliberately has **no single fixed web domain**: it binds on `0.0.0.0:5000` and is reached over the LAN by IP (`10.19.172.47` / `192.168.29.85`) or via same-origin on whatever host serves it. That keeps the API surface behind one origin and avoids CORS exposure, and lets a university run it on intranet hosts before attaching a public DNS name.

All third-party traffic is **explicit and narrow**: certificate blobs go to Pinata/IPFS, hash anchors to the Sepolia RPC, and one-time passwords to a single configured SMS domain. Student personal data never leaves the local PostgreSQL instance; the public verify endpoint returns only a validity flag plus the CID.

For a production rollout the recommendation is to front the server with a real domain and TLS (e.g. `portal.<university>.edu` → reverse proxy → `:5000`), keep the DB and SMS gateway on the private network, and rotate the Pinata/Sepolia credentials. Until then, the IP-based hosts above are the authoritative “domains” in use.

1. Admin Authentication Flow

```
Admin UI ──POST /api/admin/login──▶ Express
   │ body: { username, password }
   ├─▶ loginAttempts (in-memory Map, IP key) ── 429 if rate limit hit
   ├─▶ SELECT * FROM users WHERE username = ?
   ├─▶ verifyPass(): old sha256 OR scrypt (auto-migrate to scrypt on success)
   ├─▶ createSession() → INSERT app_sessions → token
   ├─▶ INSERT activity (ADMIN_LOGIN) + monitor_events (info)
   └─▶ 200 { token, role, permissions[], departments[] } ──▶ localStorage admin_token / admin_user

Every protected call: fetch('/api/...') + headers   x-admin-user   x-auth-token
   requireAdmin ── isAdminRequest() ──▶ DB session lookup ──▶ 200 | 403 "Admin access required"
   requirePerm('dpia'|'results'|'issues'|'monitor') ──▶ role admin OR permissions[] contains perm
```

Domain users & role mapping (`users` table)

Which domain user gets which access — the login response carries `role`, `permissions[]` and `departments[]`, and every API call is authorized against them.

USERNAME	ROLE (DOMAIN)	PERMISSIONS[]	DEPARTMENTS[] (SCOPE)
admin	**admin** — full access, bypasses perm checks	— (all)	— (all)
normal_user	user	issue, bulk, verify, list, activity, downloads	—
dept_head	department-head	list, results, verify, analytics, issues, activity	—
privacy_mgr	privacy-manager	verify, issues, activity, downloads	—
cybersec	cybersecurity	monitor, verify, issues, activity	—
Anupam	cybersecurity	monitor, verify, issues, activity, downloads, analytics, results	Computer Science
hr_office	hr	list, results, verify, downloads, activity	—
legal_team	legal	verify, issues, activity, downloads	—

Role domains (ROLE_KEYS): admin · user · department-head · privacy-manager · cybersecurity · hr · legal. Non-admin users are *department-scoped* via `getDeptScope()` — e.g. `Anupam` only sees Computer Science rows, while `admin` sees all departments.

2. Certificate Issuance Flow (admin)

```
1. AdminPage "Issue Certificate" form → collect student fields + file upload
2. POST /upload (multer, in-memory) → temp file → compute sha256 = cert hash
                                        |
                                        ▼
3. POST /api/certificates/blockchain { hash, name, ipfsHash }
   contract.addCertificate(hash,name,ipfsHash) ──► Sepolia RPC
          tx.wait()    ──► receipt.blockNumber        ──► return { txHash, blockNumber }
                                        |
                                        ▼
4. POST /api/certificates          { hash, name, rollNumber, course, dept, year, email, ipfsHash, txHash }
   INSERT INTO certificates ... ON CONFLICT(hash) DO UPDATE
   INSERT activity (CERTIFICATE_ISSUED, onChain:!!txHash)
                                        |
                                  ▼
5. Dashboard "Certificates Issued (7d)" = COUNT certificates by date; QR renders cert hash

Data stores crossed:
  [fs] upload blob ──► [ipfs/Pinata] content-addressed cid ──► [chain] txHash
  [db] certificates.hash UNIQUE ──► canonical record ──► [db] activity audit trail
```

3. Certificate Verification Flow (public)

```
Visitor "Verify a Certificate" → /#verify
   POST /api/certificates/blockchain? NO — read path:
GET /api/verify/:hash
  └─► SELECT hash, "ipfsHash" FROM certificates WHERE LOWER(hash)=$1
  └─► 200 { valid:true, hash, ipfsHash } | { valid:false }
  ✔ Frontend shows validity badge; stores only authoritative DB result (chain check optional via contract read)
```

4. Certificate Search & Download

```
GET /api/search?q=...   →   LOWER(hash|name|rollNumber|email) LIKE %q%   →   rows[]
GET /api/certificates/download   →   build .xlsx (xlsx)   →   stream download
GET /api/certificates   →   listCertificates()   →   table/export
```

5. Bulk Certificate Upload

```
Admin "Bulk Upload" → choose .xlsx
1. POST /api/certificates/bulk-upload (multer single)   →   parse workbook rows   →   { valid[], invalid[], summary
}
2. POST /api/certificates/bulk-save   →   for each valid row: INSERT ON CONFLICT DO UPDATE
  →   INSERT activity (CERTIFICATE_BULK_IMPORTED)
  →   return processed counts
```

6. Results Publication & Verification (OTP)

```
A) Publish: POST /api/result (requirePerm results) → INSERT results → activity
B) Bulk: POST /api/results/bulk-upload → xlsx parse → POST /api/results/bulk-save → INSERT
C) Verify (student): GET /api/results/verify (aadhaar+mobile) → aggregate semester rows
D) OTP retry path:
     POST /api/results/otp/request   →   gen 6-digit OTP → otpStore.set(roll,{hash,expires,attempts})
         ⟶ hash OTP (sha256) ⟶ send SMS via configured gateway (sms-gateway.js)
     POST /api/results/otp/verify   →   check sha256 match + TTL(5m) + attempts limit   →   allow view
```

7. Issues Workflow

```
Students / staff: POST /api/issues     →    INSERT issues (open)
Admin: GET /api/issues (requirePerm issues) → list
Admin: DELETE /api/issues/:id → remove/resolve → INSERT activity
Dashboard: counts open / resolved / rejected derived from issues.status
```

8. Dashboard & Analytics Flow

```
React Dashboard mount (fetch on load + Refresh button)
GET /api/dashboard/stats        →   totals: certificates, issued-today, departments, results, issues
GET /api/dashboard/analytics    →   series: certs/7d, dept-wise results, semester-wise results
   └⟶ aggregate SQL (COUNT/GROUP BY) ⟶ JSON series ⟶ bars/tables in AdminPage
```

9. Campus Monitor & Agent Ingest

```
External campus app ── POST /api/agent/event    (x-monitor-token)
   ├⟶ validate MONITOR_AGENT_TOKEN ── 403
   ├⟶ look up monitor_apps by app_key (enabled) ── 404 if unknown
   └⟶ insertMonitorEvent() → INSERT monitor_events
Poll loop (every MONITOR_POLL_INTERVAL s):
  pollMonitoredApps() → fetch app endpoints → HTTP ok/fail → INSERT monitor_checks
   GET /api/monitor/apps → status table; GET /api/monitor/stats → uptime aggregates
Admin CRUD: POST/PUT/DELETE /api/monitor/apps(:id) → monitor_apps (incl. ROPA/DPIA/DPDP fields)
```

10. DPIA Module Data Flow core deliverable

```
CREATE DPIA    POST /api/dpia (requirePerm dpia)
   validate title → map stage/status via workflow config → INSERT dpia_assessments
  → dpiaAudit('created') ; for each risk → INSERT dpia_risks → dpiaAudit('risk_added')

ADD RISK       POST /api/dpia/:id/risk          → INSERT dpia_risks (+ audit)
EDIT RISK      PUT   /api/dpia/risk/:riskId     → UPDATE dpia_risks (open→resolved, remediation)

WORKFLOW MVP   POST /api/dpia/:id/transition { to, comment }
  load dpia_workflow_config.stages/transitions
  validate allowed from→to transition (edge rules: requireAdmin / requireComment)
      └─► on 'admin' transition ✓ → UPDATE stage+status (dpiaStageToStatus)
  → dpiaAudit(action: created|approved|rejected|stage_changed)
  → INSERT activity (DPIA_APPROVED/DPIA_REJECTED/DPIA_STAGE_CHANGED)

DASHBOARD      GET /api/dpia/dashboard    (requireAdmin)
  └─► aggregate queries:
        totalDpias = COUNT(dpia_assessments)
        completed / open = GROUP BY status
        highRisk = COUNT high|critical
        openRisks   = COUNT dpia_risks status='open'
        overdue     = COUNT open AND due_date < now()
        thirdParty  = COUNT third_party=true
        byUnit table = GROUP BY business_unit, status
        dataCategories = unnest JSONB data_categories
        riskBreakdown = COUNT risks by level/status

REPORT         GET /api/dpia/report   (requireAdmin)     [route ABOVE /api/dpia/:id]
  └─► pure-text executive summary (same aggregates, formatted) → text/plain
       consumed by View Report (blob tab) & Export Report (.txt download)
READ/WRITE     GET /api/dpia (list) · GET /api/dpia/:id (detail incl. risks)
  PUT /api/dpia/:id    ·   DELETE /api/dpia/:id (cascade dpia_risks via FK)

DPIA dashboard feed (verified values on both hosts .71/.85):
 Total:14 · Completed 4/Open 10 · High-risk 6 · Open risks 13 · Overdue 0 · Third-party 7 · Categories 12
```

11. Settings, Brand, Users, Departments, Activity

```
Brand: GET/PUT /api/brand → brand table → logo/colors render site-wide
SMS:   GET/PUT /api/settings/sms → app_settings; POST /api/settings/sms/test
Users: GET/POST /api/users, PUT/DELETE /api/users/:username (permissions[] JSONB, departments[] JSONB)
Depts: GET/POST /api/departments, DELETE /api/departments/:name
Activity: GET /api/activity → audit view; DELETE /api/activity (clear, requireAdmin)
```

12. Frontend (SPA) Data Flow

```
App.js: window.location.hash → page state (home | verify | result | admin)
AdminPage.js (3681 lines): 13 modules, each mounts → fetch(`${API_BASE}/api/...`, { headers: authHeaders() })
    API_BASE = dev ? `http://${hostname}:5000` : "" (same-origin)
   authHeaders() = { 'x-admin-user': admin_user, 'x-auth-token': admin_token } ← localStorage
Login stores admin_user/admin_token; logout clears them
View Report: fetch('/api/dpia/report', headers) → blob → window.open(URL.createObjectURL(blob))
Export:           fetch(..., headers) → blob → a[download].click() + revokeObjectURL
```

13. Deployment & Cross-Host Data Flow

```
Host .71  : npm start (:3000 dev)          +   node server.js (:5000 prod, serves build/)
Host .85  : PM2 app "cert-app" (auto-restart) node server.js :5000
Both write to local PostgreSQL 16 (db.js recreates schema on boot if missing)
Sync path used 2026-09-30: rsync code+src+build → npm build → pg_dump .71 → pg_restore .85 (dpia_* added)
Report route fix deployed on both hosts; View Report verified end-to-end (blob tab opens)
```

```
Domains in play:  10.19.172.47:5000 (.71)   ·   192.168.29.85:5000 (.85)   ·   localhost:5432 (DB)
```

14. Failure & Safety Properties

```
Schema idempotency: CREATE TABLE IF NOT EXISTS + ALTER TABLE ADD COLUMN IF NOT EXISTS + guards make restarts safe.
DPIA FK cascade: deleting a DPIA removes its risks and audit rows via ON DELETE CASCADE.
Login throttling: in-memory per-IP attempt window → 429 lockout (15 min); OTP stored hashed (sha256) with 5-min TTL
and attempt cap.
Route-order rule: static DPIA subroutes (report, workflow, dashboard) are registered before /api/dpia/:id.
RBAC: roles (admin, dept-head, privacy-manager, cybersecurity, hr, legal) + per-permission gates on
DPIA/results/issues/monitor.
Agent security: monitor events require x-monitor-token and a registered, enabled app_key.
```